Practical 3

1. Distinct points (Point.java + Driver.java with main)

(a) Create class Point with private int x, y and a constructor.

(b) Override toString() to return “(x, y)”.

(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree

(e.g. Objects.hash(x, y)).

(d) In Driver.main: build a Point[] with some repeated coordinates.

(e) Count distinct points: for each point, use equals() to check whether it already

appeared earlier; count those that did not.

(f) Print “Distinct: N”.

Expected output: 5 points with 2 repeats → “Distinct: 3”.

Sol)

Point.java

```
import java.util.Objects;


public class Point {
    private int x;
    private int y;


    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }


    @Override
    public String toString() {
```

```
        return "(" + x + ", " + y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Point point = (Point) obj;
        return x == point.x && y == point.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Driver.java

```
public class Driver {
    public static void main(String[] args) {
        // Build Point array with 5 points and 2 repeats
        Point[] points = {
            new Point(1, 2),
            new Point(3, 4),
            new Point(1, 2), // Repeat 1
            new Point(5, 6),
            new Point(3, 4)  // Repeat 2
```

```
        };

        int distinctCount = 0;

        for (int i = 0; i < points.length; i++) {
            boolean isDuplicate = false;
            // Check if points[i] appeared earlier in the array
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) {
                    isDuplicate = true;
                    break;
                }
            }
            if (!isDuplicate) {
                distinctCount++;
            }
        }

        System.out.println("Distinct: " + distinctCount);
    }
}
```

Expercted Output

Distinct: 3

Learning Outcome

Object State Equality: Understanding how overriding `equals(Object)` allows comparison of objects based on internal data fields (x, y) rather than memory reference (==

Nested Array Searching: Practicing linear search algorithms using `equals()` to check for duplicates before counting elements.

Encapsulation & String Representation: Reinforcing the use of `private` instance variables and overriding `toString()` for readable output format `(x, y)`

2. Duplicate card check (Card.java + Driver.java)

(a) Create class Card with private String rank, suit and a constructor.

(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").

(c) Override equals()/hashCode() by rank + suit.

(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals()

to check it against the earlier cards.

(e) Print "Duplicate found: <card>" the first time a repeat appears.

Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

Sol)

Card.java

```
import java.util.Objects;


public class Card {
    private String rank;
    private String suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    @Override
    public String toString() {
        return rank + " of " + suit;
    }
```

```
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Card card = (Card) obj;
        return Objects.equals(rank, card.rank) && Objects.equals(suit, card.suit);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Driver.java

```
public class Driver {
    public static void main(String[] args) {
        // Cards to process sequentially
        Card[] incomingCards = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("10", "Diamonds"),
            new Card("Ace", "Spades"), // First duplicate
            new Card("King", "Hearts")  // Subsequent duplicate
        };
```

```
        Card[] hand = new Card[incomingCards.length];
        int count = 0;
        boolean duplicatePrinted = false;

        for (Card currentCard : incomingCards) {
            boolean isDuplicate = false;

            // Check against previously added cards in 'hand'
            for (int i = 0; i < count; i++) {
                if (currentCard.equals(hand[i])) {
                    isDuplicate = true;
                    if (!duplicatePrinted) {
                        System.out.println("Duplicate found: " + currentCard);
                        duplicatePrinted = true; // Print only the first time a repeat appears
                    }
                    break;
                }
            }

            // Add the card to the hand
            hand[count] = currentCard;
            count++;
        }
    }
}
```

Expercted Output

Duplicate found: Ace of Spades

Laerning Outcome

Sequential Processing: Learning to inspect incoming data dynamically before storing or accepting it into a data structure.

Flag Logic: Practicing the use of `boolean` flags to control event-driven output (ensuring the message prints only on the *first* detected duplicate).

Object Consistency: Reinforcing `hashCode()` contract compliance alongside `equals()` when defining identity based on multiple fields (`rank` and `suit`

3. (For Advanced Learners – home) Fractions (Fraction.java + Driver.java)

(a) Create class Fraction with private int num, den.

(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.

(c) Override toString() to return num + "/" + den.

(d) Override equals()/hashCode() using the reduced num and den.

(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.

Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).

Sol)

Fraction.java

```
import java.util.Objects;


public class Fraction {
   private int num;
   private int den;


   public Fraction(int num, int den) {
      if (den == 0) {
         throw new IllegalArgumentException("Denominator cannot be zero.");
```

```
        }

        // Handle negative signs (keep denominator positive)
        if (den < 0) {
            num = -num;
            den = -den;
        }

        // Reduce to lowest terms using GCD
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
    }

    // Helper method to calculate the Greatest Common Divisor
    private int gcd(int a, int b) {
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    @Override
    public String toString() {
        return num + "/" + den;
    }

    @Override
    public boolean equals(Object obj) {
```

```
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Fraction fraction = (Fraction) obj;
        return num == fraction.num && den == fraction.den;
    }

    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Driver.java

```
public class Driver {
    public static void main(String[] args) {
        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(2, 4);
        Fraction f3 = new Fraction(3, 6);

        // Printing automatically calls toString()
        System.out.println("f1: " + f1);
        System.out.println("f2: " + f2);
        System.out.println("f3: " + f3);

        // Verification checks
        System.out.println("\nEquality Checks:");
        System.out.println("f2.equals(f1): " + f2.equals(f1));
        System.out.println("f3.equals(f1): " + f3.equals(f1));
        System.out.println("f2.equals(f3): " + f2.equals(f3));
    }
}
```

Expectd Output

f1: 1/2

f2: 1/2

f3: 1/2

Equality Checks:

f2.equals(f1): true

f3.equals(f1): true

f2.equals(f3): true

Learning Outcome

Canonical Form (Invariants): Learning how to enforce business logic and object invariants directly in the **constructor** (reducing fractions to lowest terms upon instantiation).

Algorithmic Helper Methods: Implementing mathematical helper routines like the **Euclidean GCD algorithm** (`gcd(a, b)`) within an object-oriented class.

Simplified Value Objects: Observing how maintaining a standard internal form makes equality checks (`equals()`) straightforward, avoiding complex cross-multiplication formulas at comparison time.